

Tartalomjegyzék

1. Feladat kiírása	2
2. Program felépítése	3
2.1. main.py.....	3
2.2. add_noise.py.....	3
2.3. analyse_img.py.....	3
2.4. analyse_thinning.py.....	3
2.5. show_thinning.py	3
3. Kép feldolgozása.....	4
4. Algoritmus tesztelés	6
4.1. palcika1 és palcika2 – alap tesztesetek – ideális körülmények.....	7
4.2. palcika3 - világos háttér, sötét objektumok	13
4.3. palcika 4-5-6 - összetett elrendezések és kapcsolódások	15
4.4. palcika 7 – színbeli különbségek hatása.....	20

Pálcikák számolása

1. Feladat kiírása

Írjunk olyan programot, amely a palcika1.jpg ill. a palcika2.jpg képen levő pálcikákat detektálja, megszámlálja, és a kapott számot kiírja! A detektált pálcikákat jelenítse is meg jól kivehető - például vörös színű – vonalak berajzolásával! A könnyebb ellenőrizhetőség végett javasolt ezeket a vonalakat egyesével megjeleníteni, például úgy, hogy a program egy-egy újabb vonal rajzolása előtt egy-egy billentyű lenyomásáig várakozzon. A programnak egyszerre csak az egyik képet kell vizsgálnia, de mindkét képre helyesen kell működnie. (A palcika1.jpg képen 6, a palcika2.jpg képen pedig 9 db pálcika található.) Komplexebb megoldás: különböző hosszúságú és színű pálcikák számolása.



2. Program felépítése

Forrás kód:

https://github.com/nemeth-balazs/Digital_ImageProcessing_CountSticks.git

A projekt célja a képen található pálcikák számának meghatározása digitális képfeldolgozási módszerekkel, az OpenCV és a NumPy könyvtárak használatával.

A kód több modulra van bontva, ahol minden fájl egy jól elkülönített feladatért felel.

2.1. main.py

A program belépési pontja.

Feladata a teljes feldolgozási folyamat vezérlése: kép beolvasása, zaj hozzáadása, előfeldolgozás, thinning, majd az eredmények kiértékelése és megjelenítése.

2.2.add_noise.py

A kép zajosításáért felelős modul.

Tartalmaz függvényeket só-bors és Gauss zaj generálására, valamint egy egységes interfészt (`create_img_with_noise`) a zaj hozzáadására a felhasználó által megadott zajszint alapján.

2.3.analyse_img.py

Egyszerű képfeldolgozási segédfüggvényeket tartalmaz.

Jelenleg egy bináris kép invertálását megvalósító függvény található benne, amely a háttér és objektum felcserélésére szolgál.

2.4.analyse_thinning.py

A thinning eredményének kiértékeléséért felel.

Itt történik a végpontok detektálása, ezek vizualizálása, valamint a pálcikák számának meghatározása a skeleton alapján.

2.5.show_thinning.py

A vizualizációs funkciókat tartalmazza.

Lehetővé teszi a detektált pálcikák megjelenítését csoportonként vagy egyszerre, segítve az algoritmus működésének lépésenkénti ellenőrzését.

3. Kép feldolgozása

A `main.py` a teljes feldolgozási folyamatot vezérli, a kép beolvasásától egészen a pálcikák számának meghatározásáig. A lépések egymásra épülnek, minden egyes feldolgozási fázis előkészíti a következőt.

A program elején kiválasztásra kerül a bemeneti kép, majd a felhasználó megad egy zajszintet, amelyet a továbbiakban a kép módosítására használunk. Ez lehetővé teszi, hogy különböző zajviszonyok mellett is kipróbáljuk az algoritmust.

```
noise_level = add_noise.get_noise_level()
```

Ezután a kép beolvasása történik meg színes formátumban:

```
im_src = cv2.imread(img_source, cv2.IMREAD_COLOR)
```

A beolvasott képet rögtön meg is jelenítjük, hogy látható legyen a kiindulási állapot. A következő lépésben mesterséges zajt adunk hozzá:

```
img_noisy = add_noise.create_img_with_noise(im_src, noise_level)
```

A zaj hozzáadása azért fontos, mert a későbbi feldolgozás így jobban közelíti a valós körülményeket, ahol a bemenet ritkán tökéletes.

A színes képet ezután szürkeárnyaltos formára alakítjuk:

```
img_gray = cv2.cvtColor(img_noisy, cv2.COLOR_BGR2GRAY)
```

Erre azért van szükség, mert a további lépések (különösen a küszöbölés) egycsatornás képen egyszerűbben és stabilabban működnek.

A zaj csökkentése két lépésben történik. Először Gauss-szűrést alkalmazunk:

```
img_filtered = cv2.GaussianBlur(img_gray, (9, 9), sigmaX=3.0, sigmaY=3.0)
```

Ez kisimítja a képet és csökkenti a finomabb zajokat. Ezt követi egy medián szűrés:

```
img_filtered = cv2.medianBlur(img_filtered, ksize=15)
```

A medián szűrés különösen hatékony a só-bors zaj eltávolításában, miközben az élek viszonylag jól megmaradnak.

A szűrt képen ezután küszöbölést végzünk az Otsu-módszerrel:

```
_, img_otsu = cv2.threshold(img_filtered, thresh=0, maxval=255 type =  
cv2.THRESH_BINARY + cv2.THRESH_OTSU)
```

Ennek eredményeként egy bináris képet kapunk, ahol a pálcikák fehérek, a háttér pedig fekete (ideális esetben). Mivel ez nem minden képnél teljesül, egy ellenőrzés után szükség esetén invertáljuk a képet:

```
inverted, img_otsu = analyse_img.invert_img(img_otsu)
```

Ez biztosítja, hogy a további feldolgozás egységes bemeneten történjen.

A következő lépés a skeletonizáció (thinning), ahol az alakzatokat egypixeles vázra redukáljuk:

```
img_thin=cv2.ximgproc.thinning( img_otsu, thinningType =  
cv2.ximgproc.THINNING_ZHANGSUEN)
```

Ez a lépés kulcsfontosságú, mert a pálcikák középvonalát adja meg, amelyen már egyszerűbb topológiai műveleteket végezni.

A thinning eredményét több módon is megjelenítjük. Egyrészt lépésenként:

```
show_thinning.show_sticks_group(img_otsu, img_thin)
```

Másrészt egyben is:

```
img_overlay = show_thinning.show_sticks_all_groups(img_otsu, img_thin)
```

Ez segít ellenőrizni, hogy a detektált vonalak megfelelnek-e az elvárásoknak.

A következő lépés a végpontok meghatározása és megjelenítése:

```
img_end_point = analyse_thinning.get_overlay_with_end_point_mask (img_otsu,  
img_thin)
```

A végpontok a skeleon olyan pontjai, amelyeknek pontosan egy szomszédjuk van, és ezek kulcsszerepet játszanak a darabszám meghatározásában.

Végül a pálcikák számát a végpontok alapján határozzuk meg:

```
num_endpoints = analyse_thinning.get_number_of_sticks(img_thin)
```

A jelenlegi megközelítés szerint a pálcikák száma a végpontok számának fele. Ez egyszerű és sok esetben jól működik, de bizonyos geometriai elrendezések esetén korlátai vannak.

A program végén az eredményt kiírjuk, majd bezárjuk a megnyitott ablakokat.

4. Algoritmus tesztelés

Az algoritmus működését több különböző tesztképen vizsgáltuk, amelyek eltérő elrendezéseket és zajviszonyokat tartalmaznak. A cél nemcsak a helyes működés igazolása volt, hanem annak feltárása is, hogy milyen esetekben és miért ad hibás eredményt a módszer.

A jelenlegi megközelítés a skeletonizált képen meghatározott végpontokra épül, és feltételezi, hogy egy pálcikának két végpontja van. Ez az egyszerű modell sok esetben jól működik, különösen akkor, ha a pálcikák nem érintkeznek egymással vagy csak metszik egymást. Ugyanakkor bizonyos geometriai elrendezések esetén ez a feltételezés nem teljesül, ami hibás darabszámhoz vezet.

Problémát okoznak azok az esetek, amikor több pálcika egy közös pontban találkozik, vagy egymáshoz kapcsolódva egyetlen összefüggő struktúrát alkotnak. Ilyenkor a skeleton nem tartalmaz elegendő információt annak eldöntésére, hogy a kapott alakzat egyetlen hajlított objektum, vagy több különálló pálcika összessége. Emiatt az algoritmus jellemzően alulbecsli a pálcikák számát.

További nehézséget jelent a zaj jelenléte és az előfeldolgozás hatása. A zaj és a szűrés kombinációja torzíthatja az éleket, megszakíthatja a vonalakat, illetve hamis elágazásokat hozhat létre. Ezek a hibák közvetlenül befolyásolják a skeleton szerkezetét, így a végpontok detektálását is.

A következő alfejezetekben a konkrét tesztképeken keresztül mutatjuk be az algoritmus működését, valamint azokat az eseteket, ahol a fenti korlátok miatt pontatlan eredmény adódik.

4.1. palcika1 és palcika2 – alap tesztesetek – ideális körülmények

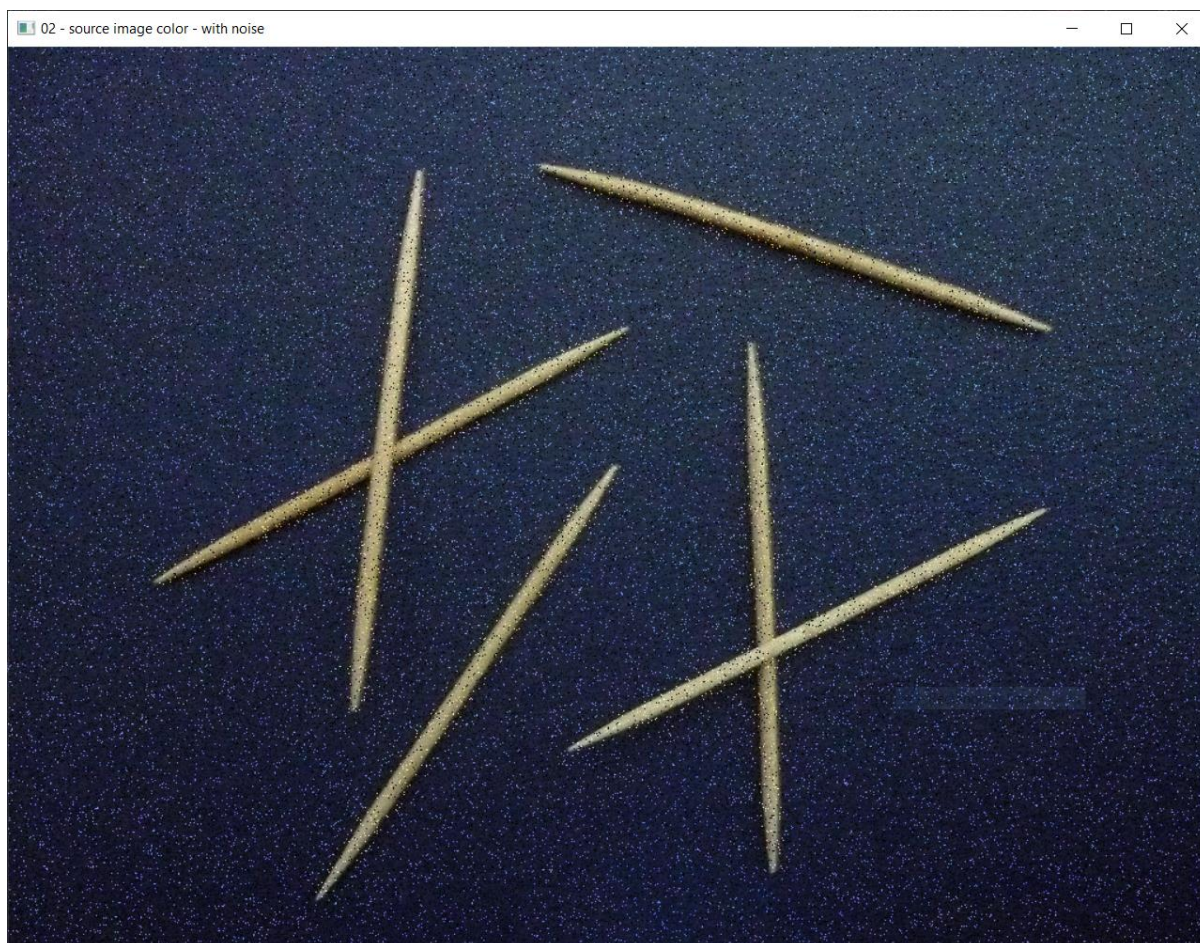
Az első két tesztkép az algoritmus működésének ellenőrzésére szolgál ideális körülmények között. Ezeken a képeken a háttér sötét, míg a pálcikák világosak, így a kontraszt jól definiált. Ennek köszönhetően a küszöbölés egyértelműen szétválasztja az objektumokat a háttértől, és a bináris kép minősége megfelelő.

A képeken a pálcikák egymást keresztezhetnek, azonban nem osztoznak közös végpontokon, így a skeletonizáció után minden pálcika két jól elkülöníthető végponttal rendelkezik. Ez megfelel az algoritmus alapfeltevésének, miszerint egy pálcika két végponttal írható le.

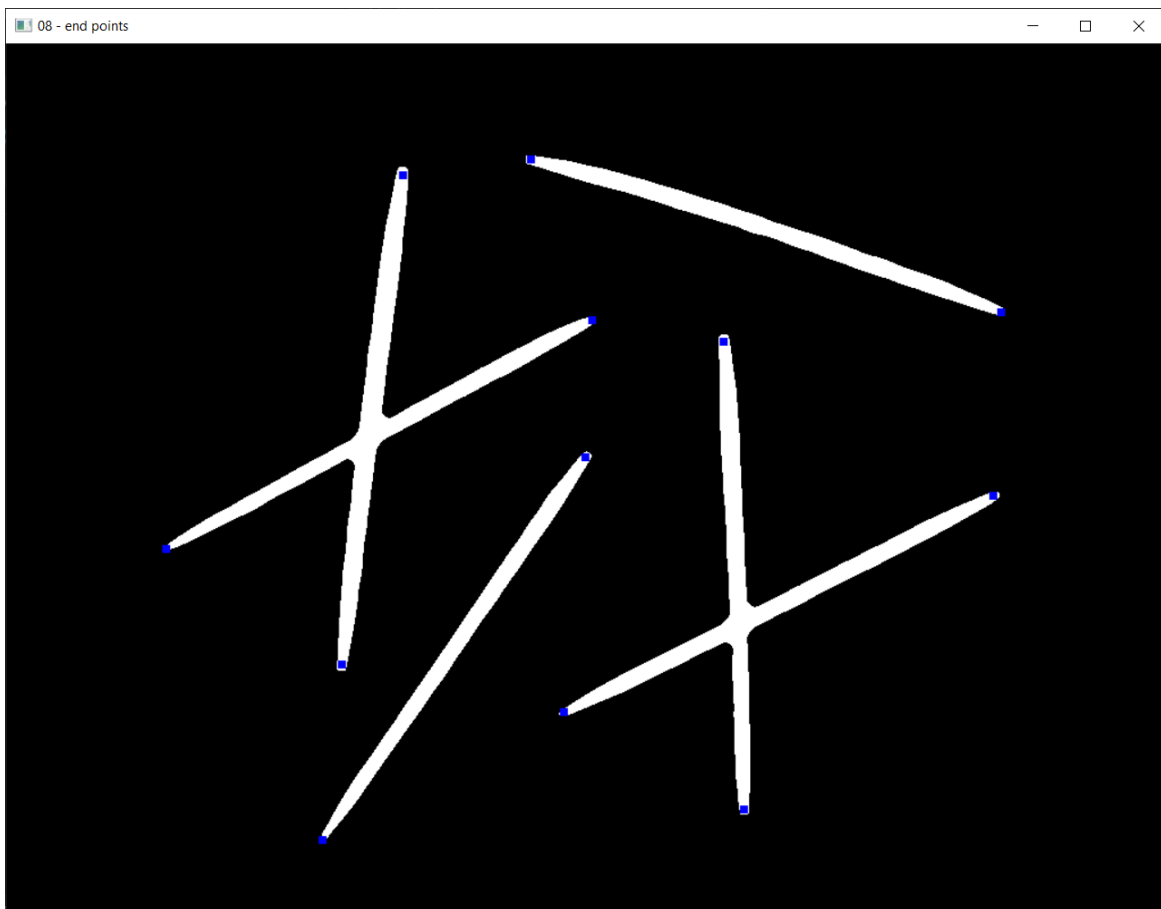
A zajszint 1 és 5 közötti értékekkel lett vizsgálva. A megfelelően megválasztott szűrési lépések (Gauss- és medián szűrés) mellett az algoritmus minden esetben helyes eredményt adott, a zaj nem befolyásolta érdemben a végpontok detektálását.

Összességében ezek a tesztesetek azt mutatják, hogy az algoritmus stabilan és megbízhatóan működik olyan környezetben, ahol a kontraszt megfelelő, és a pálcikák csak metszik egymást, de nem alkotnak összetett kapcsolódási struktúrákat.

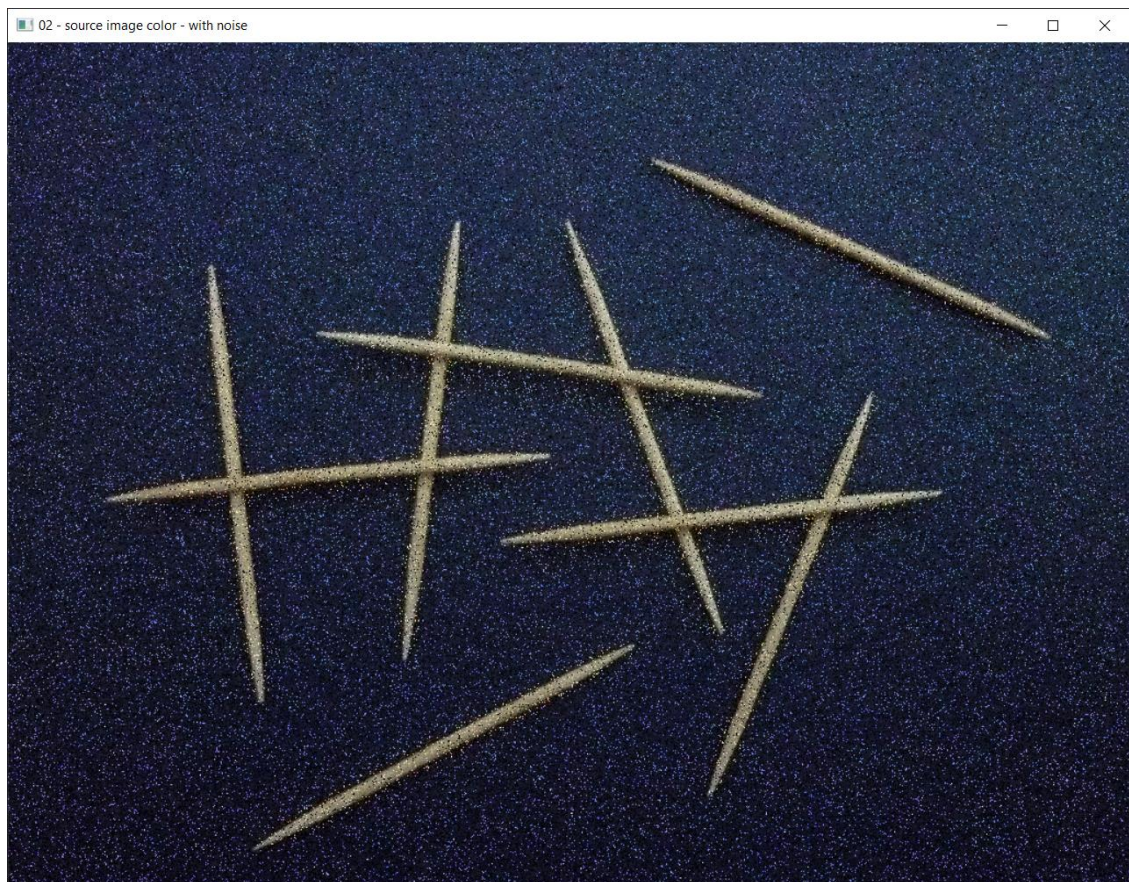
Palcika 1 - zaj szint: 3 – pálcák száma: 6

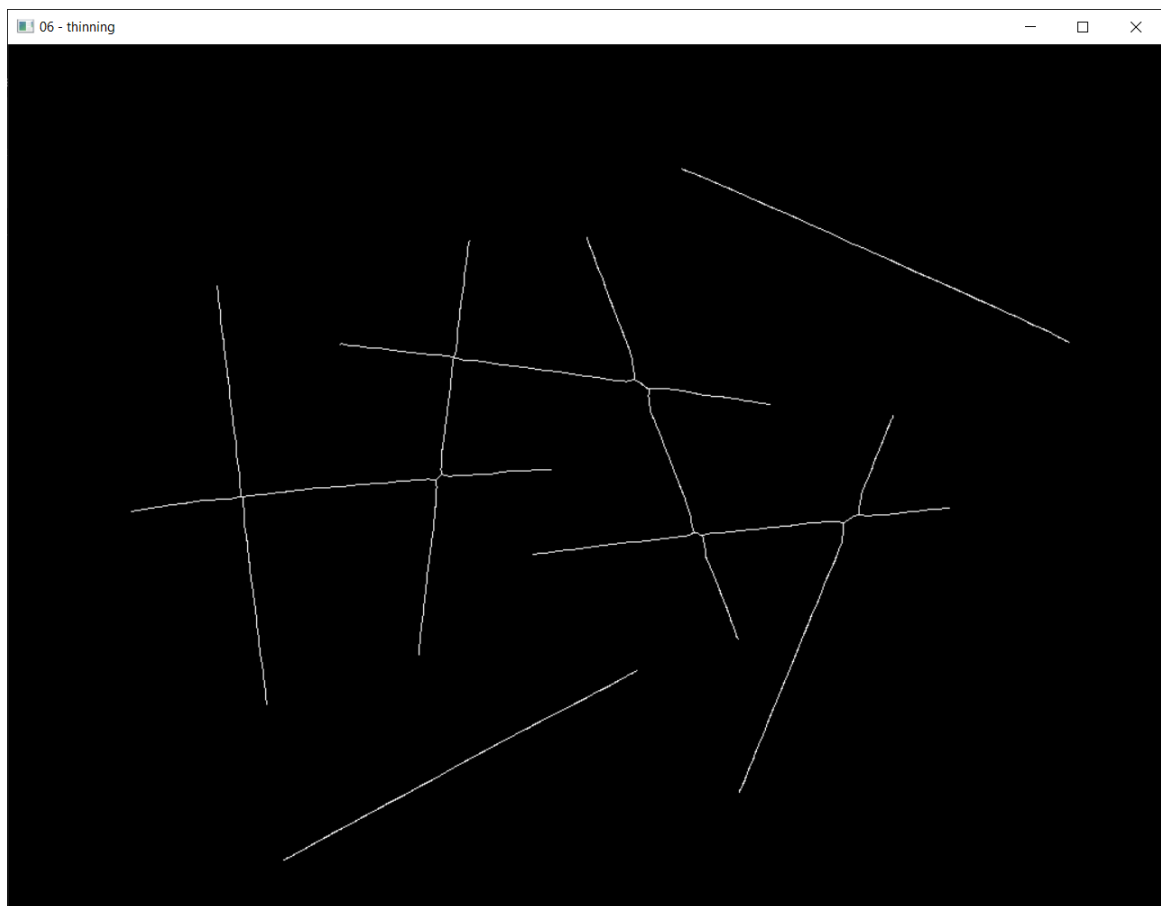


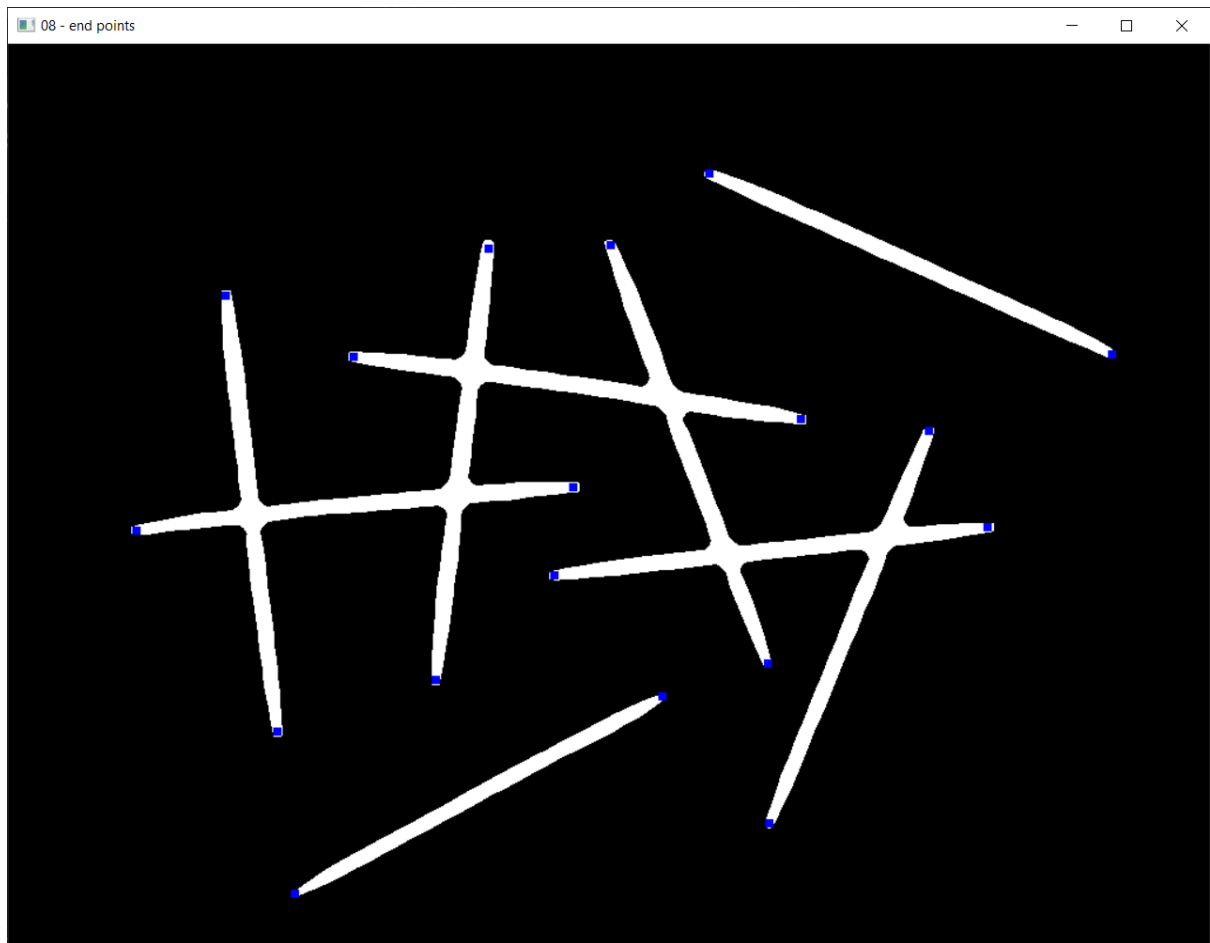




Pálcika 2 - zaj szint: 5 – pálcák száma: 9







4.2. pálcika3 - világos háttér, sötét objektumok

A harmadik tesztkép egy saját készítésű minta, amely eltér az előzőektől abban, hogy a háttér világos, míg a pálcikák sötétek. Ez a felállítás ellentétes az algoritmus által alapértelmezetten feltételezett esettel, ahol a pálcikák világosak és a háttér sötét.

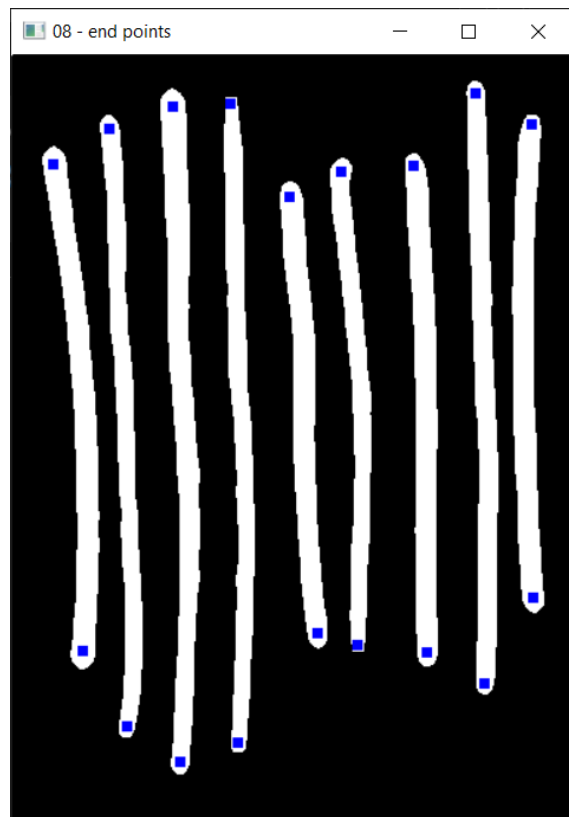
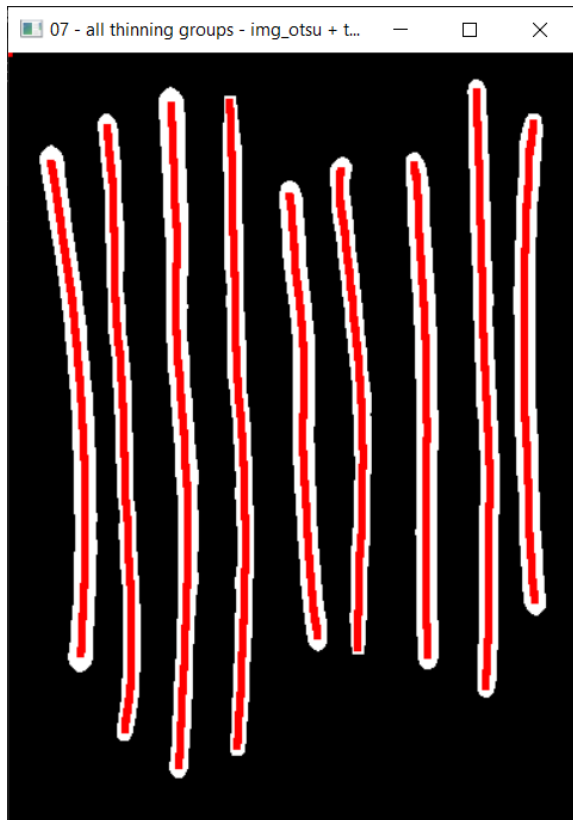
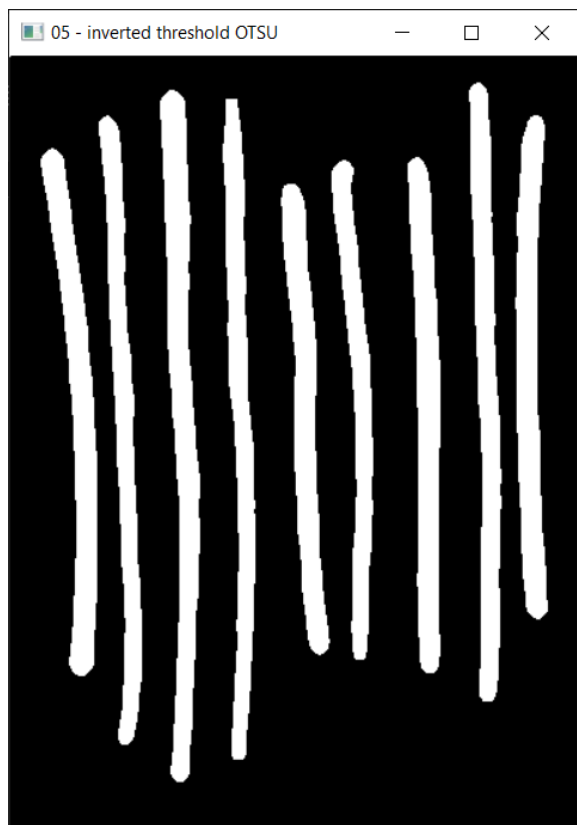
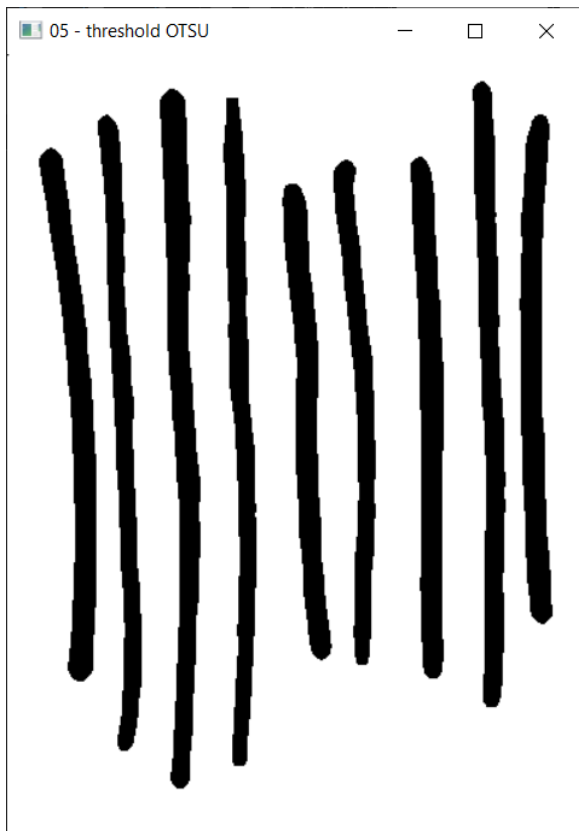
A helyes működés érdekében a bináris kép előállítás után szükség van a kép invertálására, hogy a pálcikák fehéreként jelenjenek meg a fekete háttéren. Ezt a lépést a program automatikusan elvégzi, így a további feldolgozás már egységes formátumú bemeneten történik.

A képen párhuzamos, egymástól jól elkülönülő pálcikák találhatók, amelyek nem metszik és nem is érintik egymást. Ennek megfelelően a skeletonizáció után minden pálcika két jól definiált végponttal rendelkezik, így a végpont alapú darabszám-meghatározás ebben az esetben is pontos eredményt ad.

Ez a teszt eset azt mutatja, hogy az algoritmus megfelelő előfeldolgozás (invertálás) mellett különböző fényviszonyok esetén is megbízhatóan alkalmazható.

Pálcika 3 - zaj szint: 2 – pálcák száma: 9





4.3. pálcika 4-5-6 - összetett elrendezések és kapcsolódások

A negyedik, ötödik és hatodik tesztképek már olyan eseteket tartalmaznak, ahol az algoritmus nem ad helyes eredményt. Ezek a példák jól szemléltetik a módszer korlátait.

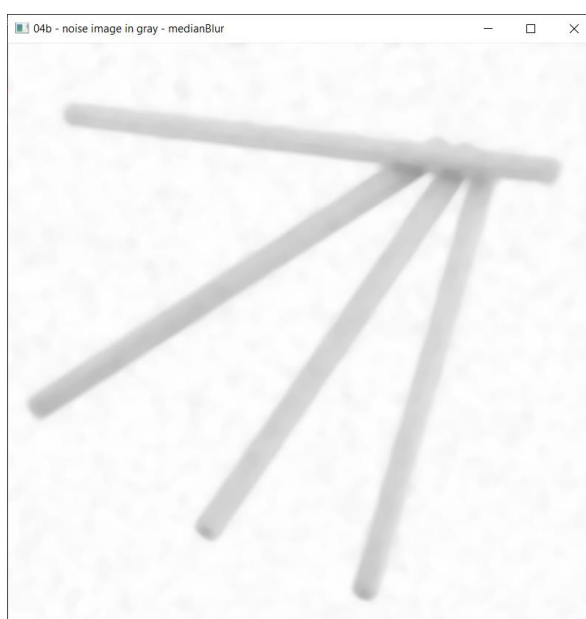
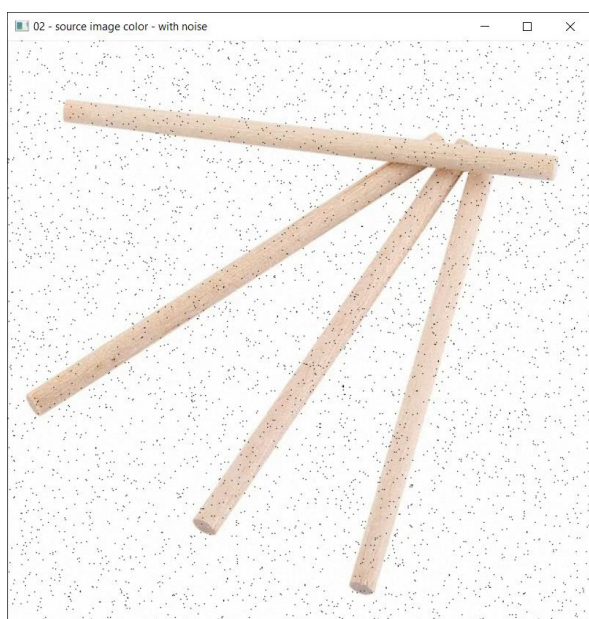
Az egyik probléma abból adódik, hogy a pálcikák vastagabbak, illetve nagyon közel helyezkednek el egymáshoz. A küszöbölés és a szűrés hatására ezek az elemek összefüggő régióvá olvadhatnak össze, így a skeletonizáció után már nem különálló pálcikák jelennek meg, hanem egyetlen összefüggő struktúra. Ebben az esetben a végpontok száma csökken, ami a darabszám alulbecsléséhez vezet.

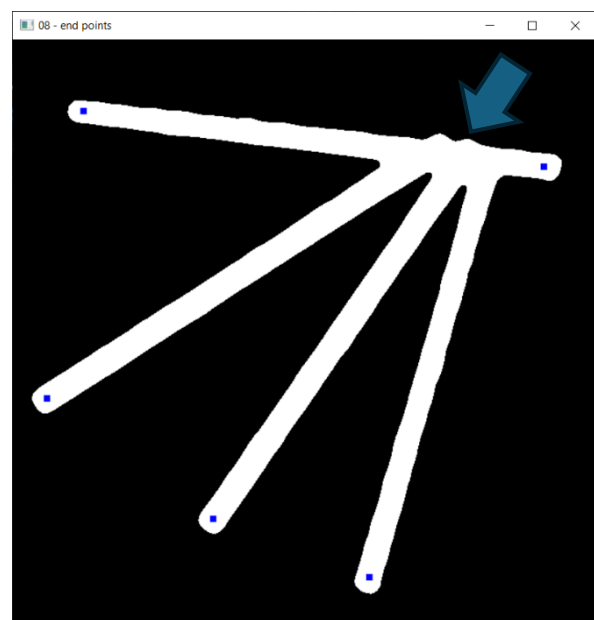
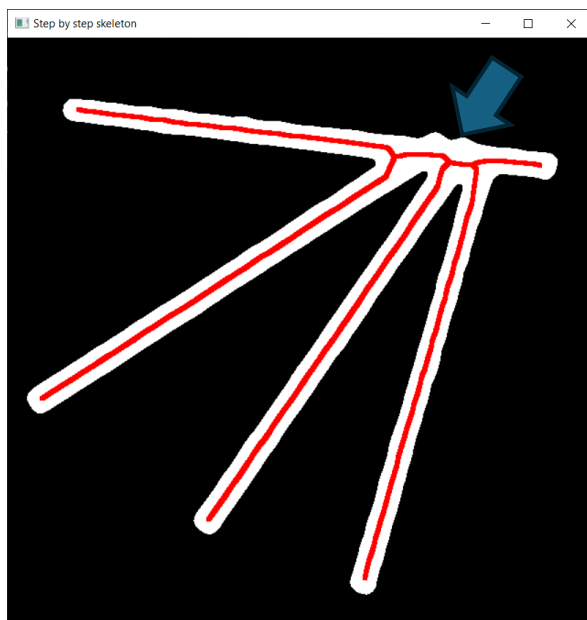
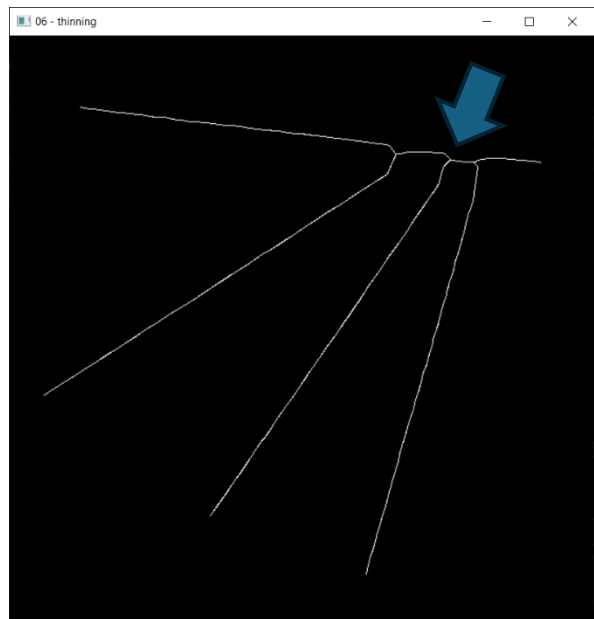
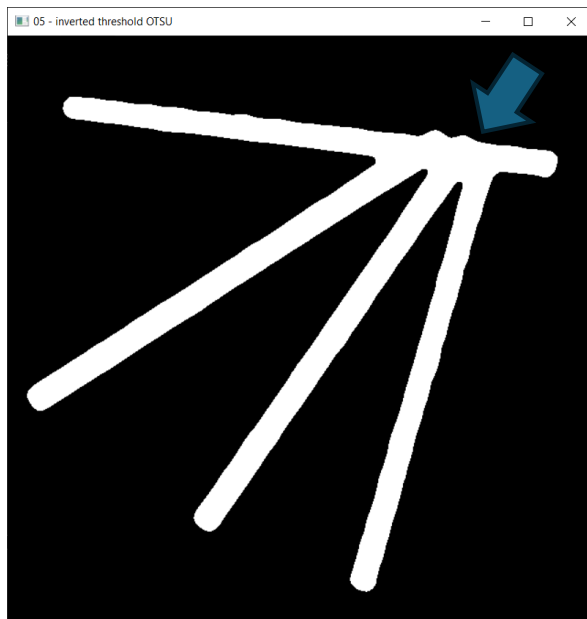
További problémát jelentenek azok az elrendezések, ahol a pálcikák végei egy közös pont közelében helyezkednek el, és „T” alakú kapcsolódásokat hoznak létre. Ilyenkor a skeleton nem minden pálcikához rendel külön végpontokat, mivel több elem egyetlen csomópontban találkozik. Ez sérti az algoritmus alapfeltevését, miszerint minden pálcikának két végpontja van.

Hasonló hibát eredményez az az eset is, amikor a pálcikák végei nem egy közös pontban találkoznak, de közel vannak egymáshoz és azonos irányba tartanak. A szűrés és a binarizálás hatására ezek a végek összekapcsolódhatnak, így a skeletonizáció már egyetlen folyamatos vonalként kezeli őket. Ennek következtében a különálló pálcikák nem különülnek el, és a számolt darabszám ismét alacsonyabb lesz a valós értéknél.

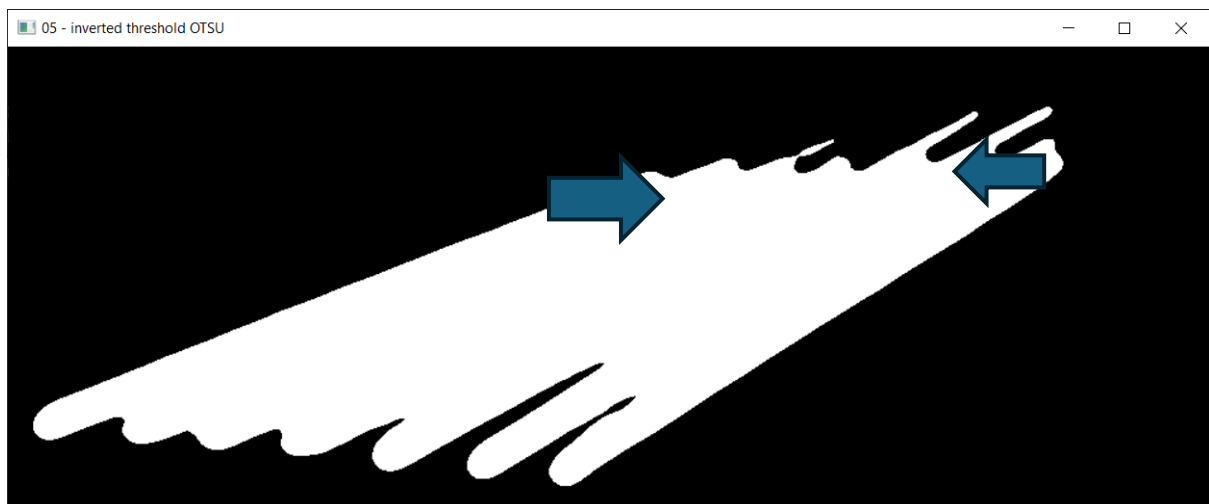
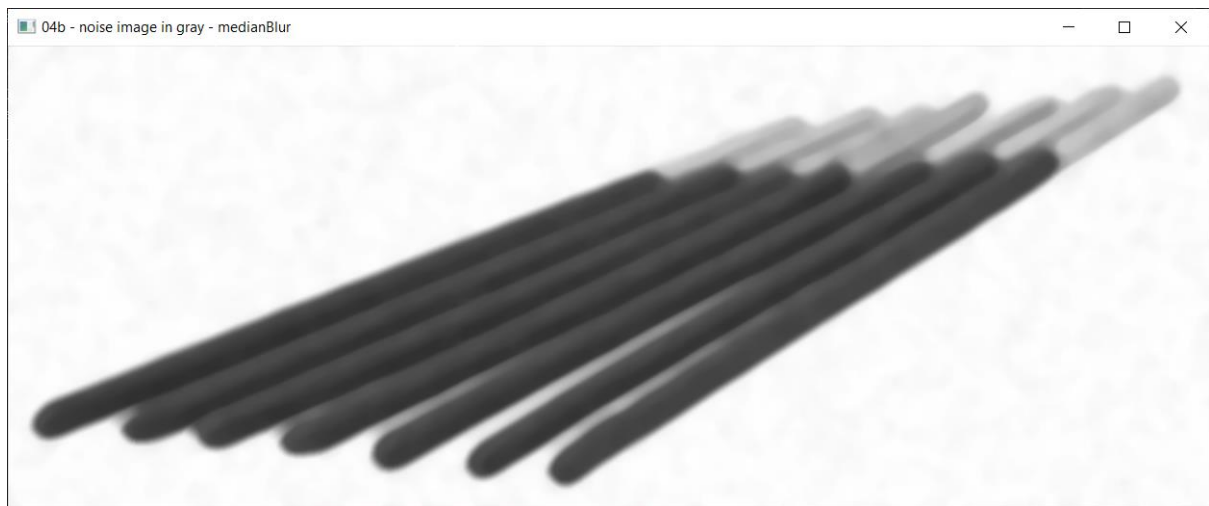
Ezek a tesztesetek jól mutatják, hogy az algoritmus érzékeny a geometriai elrendezésre, valamint az objektumok közötti távolságra, és bizonyos konfigurációk esetén nem képes helyesen szétválasztani az egyes pálcikákat.

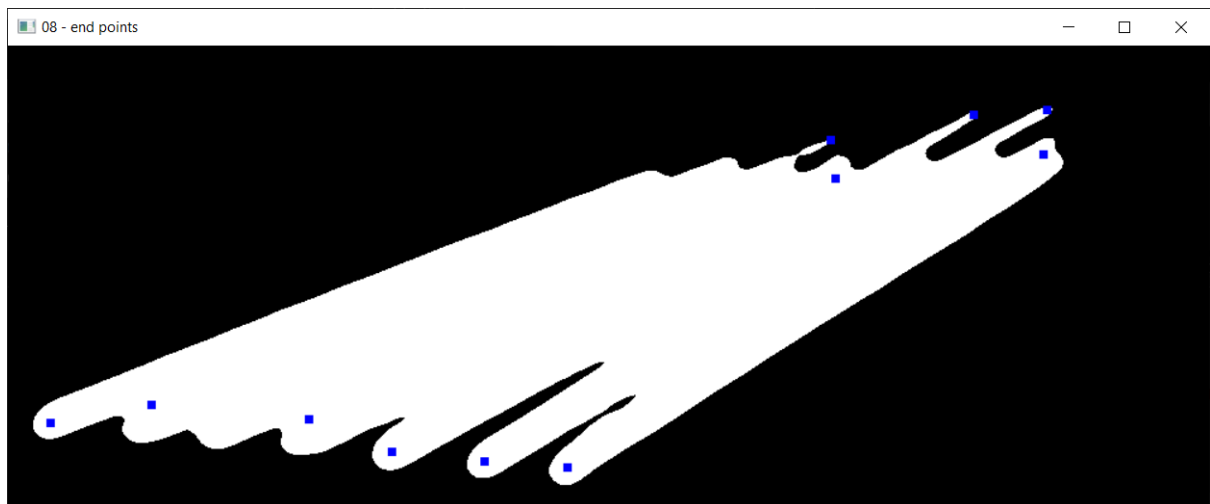
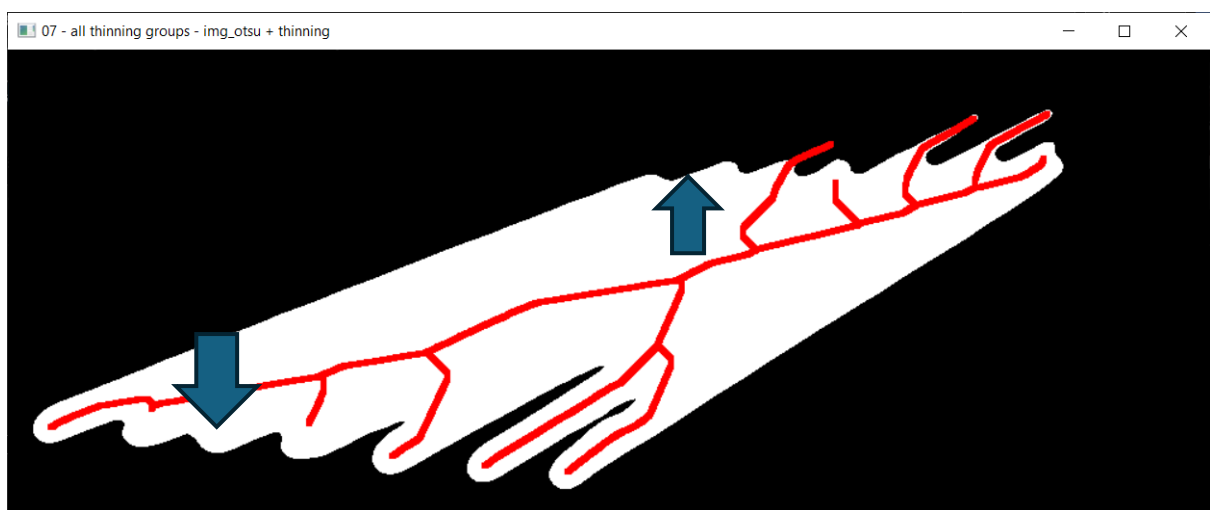
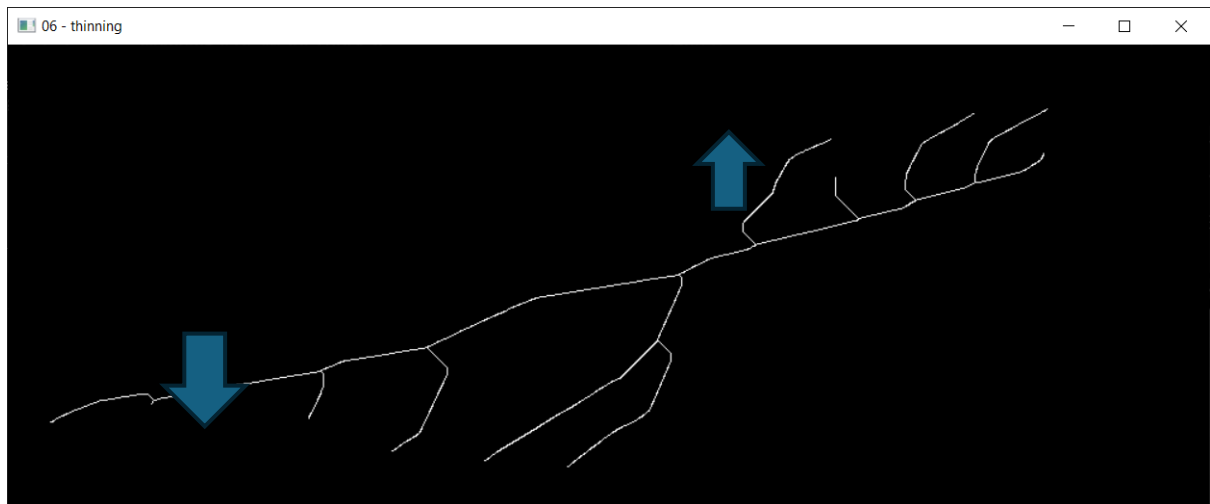
Pálcika 4 - zaj szint: 1 – „T” csatlakozás – hibás pálcika darabszám



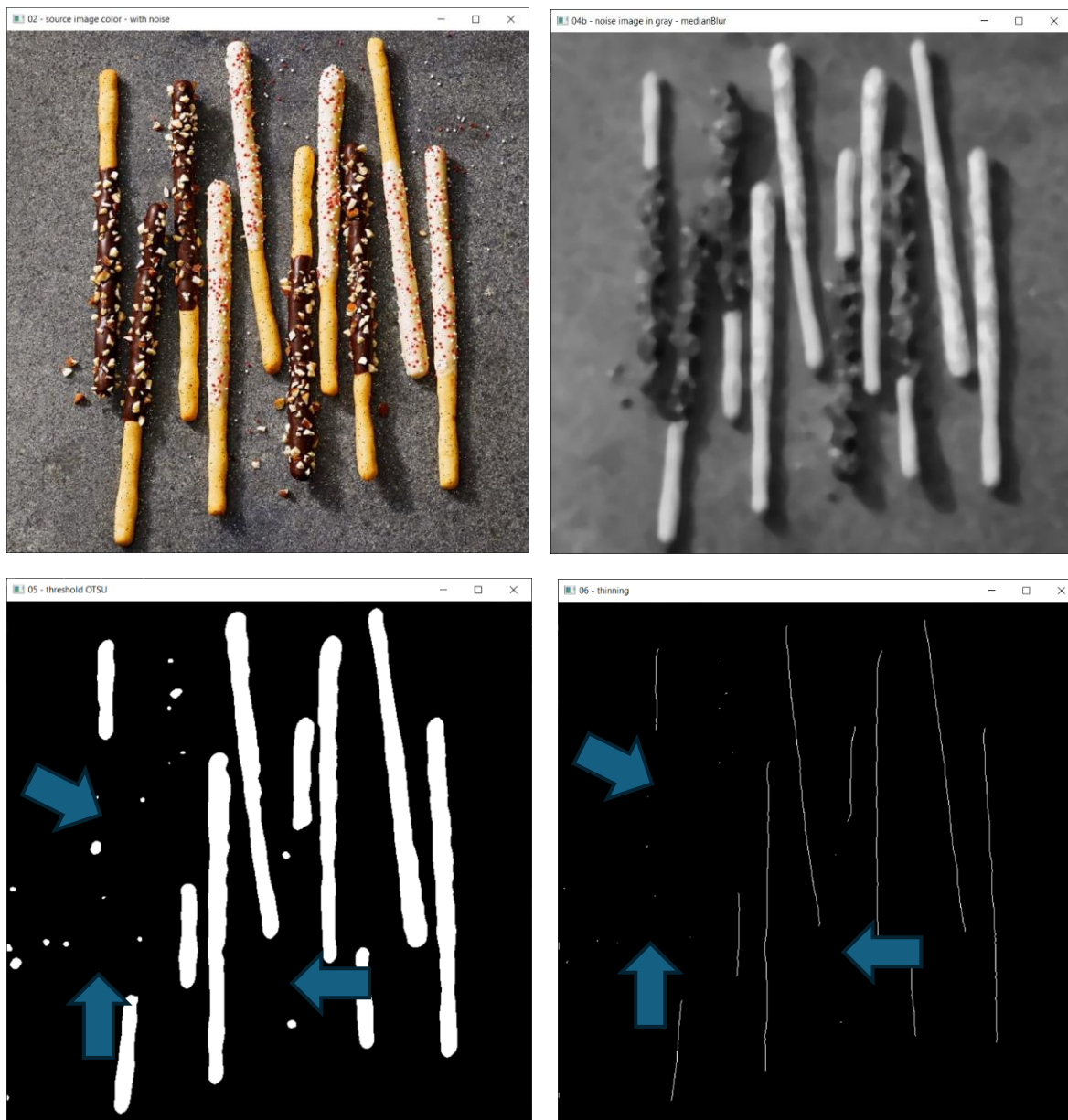


Pálcika 4 - zaj szint: 1 – egyirányba tartó pálcika végek – hibás pálcika darabszám





Pálcika 6 - zaj szint: 1 – vastag pálcikák dekorációval – hibás pálcika darabszám



4.4. pálcika 7 – színbeli különbségek hatása

A hetedik tesztkép egy mesterségesen létrehozott példa, amely ideális geometriai elrendezést tartalmaz: a pálcikák vékonyak, és kizárólag „X” alakú kereszteződések fordulnak elő. Topológiai szempontból tehát ez egy olyan eset, ahol az algoritmusnak jól kellene működnie.

A kép azonban eltér a korábbiaktól abban, hogy a háttér szürke, a pálcikák pedig különböző színekben jelennek meg. A feldolgozás során a kép szürkeárnyaltosra konvertálása után ezek a színek különböző intenzitásértékekké alakulnak. Ennek következtében a küszöbölés során egyes pálcikák – különösen a narancssárga színűek – nem különülnek el megfelelően a háttértől, és részben vagy teljesen eltűnnek a bináris képből.

Mivel a további lépések (thinning és végpont detektálás) már ezen a bináris képen alapulnak, a hiányzó vagy torzult objektumok miatt a pálcikák száma hibásan kerül meghatározásra.

Ez a teszt eset rávilágít arra, hogy az algoritmus érzékeny a színinformáció elvesztésére, és a szürkeárnyaltos konverzió után a kontraszt nem minden esetben marad elegendő az objektumok megbízható elkülönítéséhez.

Pálcika 7 - zaj szint: 1 – színes pálcikák – hibás pálcika darabszám

